SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4, BL5)

Scenario-Based: 20%

Total Marks: 50

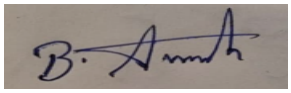
Annexure A

Code of Conduct:

I Avinash B (192511193)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Question 1: Secure Password Storage in Web Applications

Scenario: A web application stores user passwords and needs to protect them against database breaches.

(A) Why storing plaintext passwords is insecure

If passwords are stored in plaintext, anyone who gains access to the database — whether through a breach, an SQL injection attack, a stolen backup, or even a curious insider/administrator — can read every user's password immediately, with no additional effort required. This is especially dangerous because:

- Password reuse: most users reuse the same password across multiple sites, so a single leaked database can be used to break into a user's email, banking, or social media accounts (credential stuffing).
- No delay or barrier: unlike a hashed password, a plaintext password requires no cracking effort at all — the breach and the compromise happen at the same instant.
- Insider risk: employees or administrators with database access can view user passwords directly, which is both a privacy violation and a compliance failure (e.g., under GDPR or similar regulations).

(B) A secure method for storing passwords, and justification

Passwords should be stored using a salted, adaptive (slow) cryptographic hash function such as bcrypt, scrypt, or Argon2 — never the raw password and never a fast general-purpose hash alone (e.g., plain MD5 or SHA-1).

- Hashing (one-way function): the server stores only the hash of the password, never the password itself, so even a full database leak does not directly reveal user passwords.
- Salting: a unique random salt is generated per user and combined with the password before hashing. This ensures that even if two users choose the same password, their stored hashes differ, and it defeats precomputed rainbow-table attacks.
- Adaptive/slow hashing (bcrypt/scrypt/Argon2): these functions are deliberately slow and tunable (via a configurable work/cost factor), which makes brute-force and dictionary attacks computationally expensive, unlike fast hashes such as MD5/SHA-1 that can be tried billions of times per second on modern GPUs.

Together, salting and adaptive hashing ensure that even if the password database is stolen, recovering the original passwords is computationally impractical for an attacker.

Question 2: Digital Signature for Document Authentication

Scenario: A company wants to ensure that official documents sent electronically are authentic and unaltered.

(A) How digital signatures can be used in this scenario

- The sender computes a cryptographic hash (message digest) of the document.
- The sender encrypts this hash using their own private key — this encrypted hash is the digital signature, and it is attached to (or sent along with) the document.
- The recipient decrypts the received signature using the sender's public key, recovering the original hash value.
- The recipient independently recomputes the hash of the received document and compares it with the decrypted hash. If the two match, the document is confirmed authentic and unaltered.

(B) How this method ensures integrity and non-repudiation

Integrity: any modification to the document after signing — even a single changed character — produces a completely different hash when recomputed by the recipient. Since this recomputed hash will no longer match the decrypted signature, tampering is immediately detectable.

Non-repudiation: only the sender possesses the private key used to create the signature. Since the corresponding public key successfully verifies the signature, this proves the document was signed by the sender's private key specifically. The sender therefore cannot later deny having sent or signed the document, because no one else could have produced a signature that verifies correctly under their public key.

Question 3: Preventing Replay Attacks in Online Transactions

Scenario: An online payment system is vulnerable to replay attacks where attackers resend previously captured transactions.

(A) How replay attacks occur

An attacker eavesdrops on (or otherwise intercepts) a legitimate, validly encrypted or authenticated transaction message as it travels over the network. Even without being able to decrypt or forge the message's contents, the attacker simply captures the exact byte sequence and resends ("replays") it to the server at a later time. If the server has no way to distinguish this resent message from a genuinely new one — because encryption alone only protects confidentiality, not freshness — it may process the transaction again, causing a duplicate payment, a repeated authentication, or other unauthorized repeated action.

(B) A cryptographic technique to prevent replay attacks, and justification

Include a nonce (a random, single-use value) or a timestamp with each transaction, combined with a Message Authentication Code (MAC) or digital signature covering that nonce/timestamp along with the transaction data. The server keeps track of nonces already used (or enforces a tight validity time window for timestamps) and rejects any incoming message whose nonce has been seen before or whose timestamp falls outside the acceptable window.

This works because a replayed message will carry the same nonce/timestamp as the original. Even though the message content and its MAC/signature are still cryptographically valid, the server detects that this exact nonce was already used (or that the timestamp is stale) and rejects the transaction — stopping the replay without requiring the attacker's message to be decrypted or forged in the first place.

Question 4: Secure Communication over Public Wi-Fi

Scenario: A user accesses sensitive information using a public Wi-Fi network without encryption.

(A) Risks involved in this scenario

- Eavesdropping/packet sniffing: on an unencrypted network, any nearby attacker can capture all traffic, exposing passwords, session cookies, and personal data sent in the clear.
- Man-in-the-Middle (MITM) attacks: a rogue access point or ARP-spoofing attacker can position themselves between the user and the internet, silently intercepting and possibly altering traffic.
- Session hijacking: captured session cookies/tokens can let an attacker impersonate the user on a website without needing their password.
- DNS spoofing / phishing redirection: an attacker controlling the network can redirect the user to fake websites designed to steal credentials.

(B) A recommended secure communication method, and justification

Use a VPN (Virtual Private Network) with strong encryption (e.g., WireGuard, OpenVPN, or IPsec) to tunnel all traffic from the device to a trusted VPN server, and ensure that individual services are accessed only over HTTPS/TLS.

VPN: encrypts the entire traffic stream between the device and the VPN endpoint, so anyone sniffing the public Wi-Fi sees only unreadable encrypted packets rather than the actual data being sent — this neutralizes eavesdropping and most MITM attempts on the local network.

HTTPS/TLS: provides end-to-end confidentiality, integrity, and server authentication for each individual connection, ensuring that even without a VPN, the specific website's traffic remains protected and the user can verify they are communicating with the genuine server (via the site's certificate).

Using both together gives layered protection: the VPN protects against a hostile local network, while TLS protects the actual application-level session end-to-end.

Question 5: Data Integrity in Cloud Storage

Scenario: An organization stores critical data in a cloud server and wants to ensure that the data is not altered.

(A) How hashing can be used to verify data integrity

Before uploading the data, the organization computes a cryptographic hash (e.g., using SHA-256) of the data and stores this hash value (digest) securely, separately from the cloud data itself — for example, on a local system or a trusted third party. Whenever the integrity of the cloud-stored data needs to be verified, the organization downloads the data (or asks the cloud provider to compute it) and recomputes its hash using the same algorithm. If the newly computed hash matches the originally stored hash, the data is confirmed unaltered; if the hashes differ, even a single-bit change in the data has been detected, since cryptographic hash functions are designed so that any modification to the input produces a completely different output.

(B) An additional mechanism to enhance trust, and justification

Combine hashing with a digital signature (or an HMAC keyed with a secret known only to the organization) applied to the hash value itself — i.e., the organization signs the hash with its private key before storing/comparing it.

Justification: plain hashing only detects that data has changed; it does not prove who produced the original, trusted hash value, so an attacker capable of altering both the cloud data and the separately stored hash could still evade detection. A digital signature over the hash adds authenticity and non-repudiation — anyone verifying the signature with the organization's public key can confirm that the stored hash genuinely originated from the organization and has not itself been tampered with, closing this gap and providing a stronger, verifiable chain of trust for the integrity check.